

EJERCICIOS PRACTICOS - Linux - Curso 2025 Parte II.

Requerimiento: Actividad a realizar en máquina virtual generada.

Bajar el archivo de la locación <https://ssl.smn.gob.ar/dpd/zipopendata.php?dato=regtemp>, guardarlo en una carpeta de home de usuario de la máquina Linux.

Descomprimir el archivo bajado, y obtener el archivo con extensión .TXT, mostrar.

1. Redirección de la entrada/salida

Es posible cambiar la fuente de la entrada o el destino de la salida de los comandos.

Toda la entrada y salida 'E/S' se hace a través de archivos.

En LINUX, cada proceso tiene asociados 3 archivos para la administrar la E/S:

Nombre	Descriptor de archivo	Destino por defecto
entrada estándar (stdin)	0	teclado
salida estándar (stdout)	1	pantalla
error estándar (stderr)	2	pantalla

por defecto, un proceso toma su entrada de la entrada estándar, envía su salida a la salida estándar y los mensajes de error a la salida de error estándar

Para cambiar la entrada/salida se usan los siguientes caracteres:

Carácter	Resultado
comando < archivo	<i>Toma la entrada de archivo</i>
comando > archivo	<i>Envía la salida de comando a archivo; sobrescribe cualquier cosa de archivo</i>
comando 2> archivo	<i>Envía la salida de error de comando a archivo (el 2 puede ser reemplazado por otro descriptor de archivo)</i>
comando >> archivo	<i>Añade la salida de comando al final de archivo</i>
comando << etiqueta	<i>Toma la entrada para comando de las siguientes líneas, hasta una línea que tiene sólo etiqueta</i>
comando 2>&1	<i>Envía la salida de error a la salida estándar (el 1 y el 2 pueden ser reemplazado por otro descriptor de archivo, p.e. 1>&2)</i>
comando &> archivo	<i>Envía la salida estándar y de error a archivo; equivale a comando > archivo 2>&1</i>
comando1 comando2	<i>pasa la salida de comando1 a la entrada de comando2 (pipe)</i>

Ejemplos

ls -l > lista.archivos	Crea el archivo lista.archivos conteniendo la salida de ls -l
ls -l /etc >> lista.archivos	Añade a lista.archivos el contenido del directorio /etc
cat < lista.archivos more	Muestra el contenido de lista.archivos página a página (equivale a more lista.archivos)
ls /test 2> /dev/null	Envía los mensajes de error al dispositivo nulo (a la basura)
> touch /test	Crea el archivo kk vacío
cat > entrada	Lee información del teclado, hasta que se teclea Ctrl-D; copia todo al archivo entrada
cat << END > entrada	Lee información del teclado, hasta que se introduce una línea con END; copia todo al archivo entrada
ls -l /bin/bash /test > salida 2> error	Redirige la salida estándar al archivo salida y la salida de error al archivo error
ls -l /bin/bash /test > salida.y.error 2>&1	Redirige la salida estándar y de error al archivo salida.y.error; el orden es importante:
ls -l /bin/bash /test 2>&1 > salida.y.error	no funciona, por qué?
ls -l /bin/bash /test &> salida.y.error	Igual que el anterior
cat /etc/passwd > /dev/tty2	Muestra el contenido de /etc/passwd en el terminal

	tty2
--	------

2. **SORT**

Permite ordenar líneas de los archivos de entrada a partir de criterios de ordenación. Los espacios en blanco son tomados por defecto como separadores de campo. Su sintaxis es de la forma:

```
$ sort [opciones] [archivo]
```

Algunos de sus opciones son:

- b ignorar espacios en blanco precedentes.*
- d ordenar ignorando todos los caracteres salvo caracteres letras, números y espacios.*
- f considerar iguales las mayúsculas y minúsculas.*
- n ordenar por valor numérico.*
- r invertir el orden.*
- k n1,[n2] Especifica un campo como clave de ordenación, comienza en n1 y acaba en n2; los números de campo empiezan en 1.*
- o salida.txt Escribir el resultado en salida.txt.*

3. **HEAD**

Sirve para mostrar en pantalla líneas de un archivo. Por defecto se muestran las primeras 10 líneas, pero este número puede variar dependiendo de las especificaciones del usuario. Su sintaxis es la siguiente:

```
$ head -opciones archivo
```

- n: Permite especificar el número de líneas que hay que mostrar.
- q: Evita que se muestren los títulos de los archivos especificados.
- c: Permite especificar el número de caracteres a desplegar, en vez de líneas.

4. **AWK**

AWK es una herramienta de procesamiento de patrones en líneas de texto. Su utilización estándar es la de filtrar archivos o salida de comandos de UNIX/Linux, tratando las líneas para mostrar una determinada información sobre las mismas.

El uso básico de AWK es:

```
$ awk [condicion] { comandos }
```

Donde:

[condicion] representa una condición de matcheo de líneas o parámetros.

comandos : serie de comandos a ejecutar:

\$0 → Mostrar la línea completa

\$1-\$N → Mostrar los campos (columnas) de la línea especificados.

FS → Field Separator (Espacio o TAB por defecto)

NF → Número de campos (fields) en la línea actual

NR → Número de líneas (records) en el stream/archivo a procesar.

OFS → Output Field Separator (" ").

ORS → Output Record Separator ("\n").

RS → Input Record Separator ("\n").

BEGIN → Define sentencias a ejecutar antes de empezar el procesado.

END → Define sentencias a ejecutar tras acabar el procesado.

length → Longitud de la línea en proceso.

FILENAME → Nombre del archivo en procesamiento.

ARGC → Número de parámetros de entrada al programa.

ARGV → Valor de los parámetros pasados al programa.

Las funciones utilizables en las condiciones y comandos son, entre otras:

```
close(archivo_a_reiniciar_desde_cero)
```

```
cos(x), sin(x)
index()
int(num)
length(string)
substr(str,pos,len)
tolower()
toupper()
system(comando_del_sistema_a_ejecutar)
printf()
```

Los operadores soportados por awk son los siguientes:

```
*, /, %, +, -, =, ++, --, +=, -=, *=, /=, %=.
```

El control de flujo soportado por AWK incluye:

```
if ( expr ) statement
if ( expr ) statement else statement
while ( expr ) statement
do statement while ( expr )
for ( opt_expr ; opt_expr ; opt_expr ) statement
for ( var in array ) statement
continue, break
(condicion)? a : b → if(condicion) a else b;
```

Las operaciones de búsqueda son las siguientes:

```
/cadena/ → Búsqueda de cadena.
/^cadena/ → Búsqueda de cadena al principio de la línea.
/cadena$/ → Búsqueda de cadena al final de la línea.
$N ~ /cadena/ → Búsqueda de cadena en el campo N.
$N !~ /cadena/ → Búsqueda de cadena NO en el campo N.
/(cadena1)|(cadena2)/ → Búsqueda de cadena1 OR cadena2.
/cadena1/,/cadena2>/ → Todas las líneas entre cadena1 y cadena2.
```

Ejemplos con awk.

Mostrar una determinada columna de información:

```
# ls -l | awk '{ print $5 }'
```

Mostrar varias columnas, así como texto adicional (entre comillas):

```
# ls -l | awk '{ print $8 ":" $5 }'
```

Imprimir campos en otro orden:

```
# awk '{ print $2, $1 }' archivo
```

Imprimir último campo de cada línea:

```
# awk '{ print $NF }' archivo
```

Imprimir los campos en orden inverso:

```
# awk '{ for (i = NF; i > 0; --i) print $i }' archivo
```

Imprimir última línea:

```
# awk '{line = $0} END {print line}'
```

Imprimir primeras N líneas:

```
# awk 'NR < 100 {print}'
```

Mostrar las líneas que contienen valores numéricos mayor o menor que uno dado:

```
# ls -l | awk '$5 > 1000000 { print $8 ":" $5 }'
```

```
# ls -l | awk '$5 > 1000000 || $5 < 100 { print $8 ":" $5 }'
```

Mostrar la línea con el valor numérico más grande en un campo determinado:

```
# awk '$1 > max { max=$1; línea=$0 }; END { print max, línea }' archivo
```

Mostrar aquellos archivos cuya longitud es mayor que 10 caracteres:

```
# ls -l | awk 'length($8) > 10 { print NR " " $8 }'
```

Match/Sustitución de cadenas

Mostrar las líneas que contengan una determinada cadena

```
# awk '/cadena/ { print }' archivo
```

Mostrar las líneas que NO contengan una determinada cadena

```
# awk '! /cadena/ { print }' archivo
```

5. SED (Stream Editor)

Es un editor de flujos y archivos de forma no interactiva.

Permite modificar el contenido de las diferentes líneas de un archivo en base a una serie de comandos o archivo de comandos (-f archivo_comandos).

Sed recibe por stdin (o vía archivo) una serie de líneas para manipular, y aplica a cada una de ellas los comandos que le especifiquemos a todas ellas, a un rango de las mismas, o a las que cumplan alguna condición.

Por ejemplo:

Sustituir apariciones de cadena1 por cadena2 en todo el archivo:

```
# sed 's/cadena1/cadena2/g' archivo > archivo2
```

Sustituir apariciones de cadena1 por cadena2 en las líneas 1 a 10:

```
# comando | sed '1,10 s/cadena1/cadena2/g'
```

Eliminar las líneas 2 a 7 del archivo

```
# sed '2,7 d' archivo > archivo2
```

Buscar un determinado patrón en un archivo:

```
# sed -e '/cadena/ !d' archivo
```

Buscar AAA o BBB o CCC en la misma línea:

```
# sed '/AAA/!d; /BBB/!d; /CCC/!d' archivo
```

Buscar AAA y BBB y CCC:

```
# sed '/AAA.*BBB.*CCC/!d' archivo
```

Buscar AAA o BBB o CCC (en diferentes líneas, o grep -E):

```
# sed -e '/AAA/b' -e '/BBB/b' -e '/CCC/b' -e d
```

```
# gsed '/AAA\|BBB\|CCC/!d'
```

Ejemplos de Inserción

Insertar 3 espacios en blanco al principio de cada línea:

```
# sed 's/^/ /' fichero
```

Añadir una línea antes o después del final de un fichero (\$=última línea):

```
# sed -e '$i Prueba' fichero > fichero2
```

```
# sed -e '$a Prueba' fichero > fichero2
```

Insertar una línea en blanco antes de cada línea que cumpla una regex:

```
# sed '/cadena/{x;p;x;}' fichero
```

Insertar una línea en blanco detras de cada línea que cumpla una regex:

sed '/cadena/G' fichero

Insertar una línea en blanco antes y después de cada línea que cumpla una regex:

sed '/cadena/{x;p;x;G;}' fichero

Insertar una línea en blanco cada 5 líneas:

sed 'n;n;n;n;G;' fichero

Insertar número de línea antes de cada línea:

sed = filename | sed 'N;s/\n/t/' fichero

Insertar número de línea, pero sólo si no está en blanco:

sed '/./=' fichero | sed '/./N; s/\n/ /'

Si una línea acaba en \ (backslash) unirla con la siguiente:

sed -e :a -e '\\$/N; s/\\n//; ta' fichero

Ejemplos de Selección/Visualización

Ver las primeras 10 líneas de un fichero:

sed 10q

Ver las últimas 10 líneas de un fichero:

sed -e :a -e '\$q;N;11,\$D;ba'

Ver un rango concreto de líneas de un fichero:

cat -n fich2 | sed -n '2,3 p'

2 línea 2

3 línea 3

(Con cat -n, el comando cat agrega el número de línea).

(Con sed -n, no se imprime nada por pantalla, salvo 2,3p).

Ver un rango concreto de líneas de varios ficheros:

sed '2,3 p' *

línea 2 fichero 1

línea 3 fichero 1

línea 2 fichero 2

línea 3 fichero 2

(-s = no tratar como flujo sino como ficheros separados)

Sólo mostrar la primera línea de un fichero:

sed -n '1p' fichero > fichero2.txt

No mostrar la primera línea de un fichero:

sed '1d' fichero > fichero2.txt

Mostrar la primera/ultima línea de un fichero:

sed -n '1p' fichero

sed -n '\$p' fichero

Imprimir las líneas que no hagan match con una regexp (grep -v):

sed '/regexp/!d' fichero

sed -n '/regexp/p' fichero

Mostrar la línea que sigue inmediatamente a una regexp:

sed -n '/regexp/{n;p;}' fichero

Mostrar desde una expresión regular hasta el final de fichero:

sed -n '/regexp/, \$p' fichero

Imprimir líneas de 60 caracteres o más:

```
# sed -n '/^\.{60\}/p' fichero
```

Imprimir líneas de 60 caracteres o menos:

```
# sed -n '/^\.{65\}/!p' fichero
```

```
# sed '/^\.{65\}/d' fichero
```

Actividad práctica:

Con el archivo TXT, obtener mediante combinación de comandos SORT, AWK, SED, GREP, CAT, lo siguiente:

1. Cantidad de registros tomados para CORRIENTES.
2. La cantidad de registros de una lectura diaria.
3. El número de registros SIN lectura (sea Máxima o Mínima)
4. La fecha en que se registró la mayor temperatura para CORRIENTES.
5. La menor temperatura registrada en RESISTENCIA.
6. El Promedio de temperatura máxima registrado en FORMOSA.
7. El Promedio de temperatura mínima registrado en POSADAS.
8. Los registros de temperatura obtenidos en JUJUY ordenados por la mayor temperatura, de mayor a menor valor, alojarlo en el archivo 'Jujuy-menor-a-mayor.txt'

Las salidas obtenidas almacenarlas en un único archivo de texto Lab2Linux-DNI.txt para después subirlo a Aula virtual.